Proof of Secure Element Attestation Without a Certificate Authority

Jan Kučera
admin@bismuth.cz
nadochain.com github.com/hclivess/posea
September 2026

***Abstract*—Permissionless systems that reward participants rather than capital face an adversary ordinary countermeasures cannot price: the identity farm. Every per-identity admission rule—registration work, waiting periods, address caps—costs an adversary holding n identities exactly what it costs an honest participant holding one, per identity; only the adversary amortises setup. We report a deployment where this failed totally: some $1{,}000$ of $1{,}191$ mining identities belonged to two or three operators running browser farms, capturing 42% of emission.**

Proof of Secure Element Attestation (PoSEA) admits a participant on a hardware attestation verified offline against pinned vendor roots, one identity per secure element. The obvious claim—that this makes identities *scarce* rather than expensive—is false: n devices cost n times as much, so a device farm is the ASIC of this mechanism. What changes is the constant, and in the measured case that was the whole problem: the attack was browser processes whose thousandth identity cost nothing.

Such attestation has always needed an issuer. A TPM's endorsement key is vendor-certified but cannot sign, so the signing key must be certified by a privacy CA able to assert any chip's identity—and of 664 attempts, 26.8% came from machines whose healthy TPM 2.0 that authority has no registration for. On Linux no such service exists. We remove the issuer: `TPM2_MakeCredential`'s blob is deterministic in its (seed, name, secret) triple, so a challenge issued once is recomputable offline by any verifier forever, and the possession proof needs only to be *ordered*, not signed. Four published messages, with a commit–reveal across k beacon-drawn challengers, replace the authority.

The residual assumption is vendor CA integrity, not key extraction: extraction yields one identity per chip physically held, since certifying a key the vendor never certified still needs the vendor. We complete the enrolment on physical silicon and report that the collusion bound is weaker than it first appears—an adversary can *resample* the draw, converting "all k must collude" into a grinding attack whose expected cost we quantify.

***Index Terms*—Trusted platform module, remote attestation, privacy CA, commit–reveal, Sybil resistance**

I. Introduction

Hardware attestation answers a question no software can: *is this key inside a genuine secure element?* A TPM 2.0 answers it awkwardly. The chip carries an *endorsement key* that its silicon vendor certified at manufacture, which is exactly the credential one wants—and which TCG defines as a restricted decryption key [11]. Real endorsement certificates carry `Key Usage: critical, Key Encipherment` with `digitalSignature` absent. The credential that proves the chip is genuine cannot sign anything.

Attestation therefore uses a second key, an attestation key, and needs somebody to certify that this second key lives in the same chip as the first. TCG's answer is a privacy certificate authority [10]: it runs the possession proof once, interactively, and signs a certificate that verifiers can check later. Direct Anonymous Attestation [12] removes the CA's ability to link attestations but keeps an issuer. Either way, a party exists whose key can assert that any key is in any chip.

That party is a single point of failure in the ordinary sense, and—as we measured—also in a duller one: it can simply not answer. Table II gives the outcome of 664 consecutive attestation attempts against a public deployment. 26.8% came from machines whose TPM was healthy, enabled and physically present, and whose platform produced no attestation at all, because the platform vendor's attestation service has no certificate authority registered for that chip's key identifier and answers HTTP 404. The condition is service-side, the identifiers are not registrable by end users, and no client-side change repairs it. On Linux there is no equivalent service and there never has been, so the figure there is 100%.

Those machines are not short of evidence. They hold a vendor-signed endorsement certificate; the hardware proof exists and verifies offline against a pinned root. What is missing is one statement: *this attestation key lives in that certified chip*.

A. Contributions

- We observe that `TPM2_MakeCredential`'s credential blob is deterministic in its inputs (Observation 1), which converts an inherently interactive possession proof into an artifact any verifier can re-derive offline and forever (§IV).
- Building on it, we give a TPM enrolment protocol with **no certificate authority and no long-lived key of any kind**: four published messages whose security comes from their *order* rather than from any signature. It needs only an agreed total order on published messages—a ledger, an append-only log, or any ordering service.
- We show the security reduces to publication order and to the `restricted` object attribute (§V), and that secure-element key extraction—usually treated as the assumption such schemes live by—yields one identity per chip *physically held* however far the technique generalises (§V-D).

- We report two implementation obstacles that will cost every implementer the same time: real vendor endorsement certificates are not strictly DER and the obvious library refuses them, and one vendor's published "root" is a cross-signed intermediate (§VI).
- We show that the collusion bound is weaker than it appears: because an unanswered enrolment must be abandonable for liveness, an adversary can *resample* the challenger draw, turning "all k must collude" into a grinding attack. We quantify its expected cost against a deployed weight distribution and state the tension it creates with liveness rather than claiming a fix (§IV-E).
- We validate against a TPM 2.0 simulator and complete the enrolment on a physical AMD firmware TPM on a live permissionless chain (§VII-A), reporting four measured results that each contradicted an expectation—including that opening the credential through a `PolicySecret` session costs zero dictionary-attack attempts, and that the endorsement key refuses to act as a storage parent at all.
- We report that a verifier's root set is never finished, and that an incomplete one is indistinguishable from a forged chain at validation time while silently excluding honest hardware (§VII).
- We identify the residual assumption precisely (§IX): vendor certificate authority integrity, and nothing else.

II. Background

A. TPM keys and why one of them cannot sign

A TPM 2.0 derives keys from seeds held in three hierarchies. The *endorsement key* (EK) is derived from the endorsement seed under a template fixed by TCG's EK Credential Profile [11]; because the template is fixed, the key is reproducible on demand and the vendor can certify it at manufacture. Its attributes make it `restricted`, `decrypt`, and `adminWithPolicy`. It is not a signing key.

An *attestation key* (AIK) is a `restricted` *signing* key. `restricted` is the attribute the entire field rests on: such a key signs only structures the TPM itself generated, never a message its host composed. That is what makes `TPM2_Certify` evidence rather than merely a signature.

Proving possession of a decryption-only key has exactly one shape—send it something only it can decrypt and observe it return. TPM 2.0 provides `TPM2_MakeCredential` and `TPM2_ActivateCredential` for precisely this: a challenger seals a secret to an endorsement key, bound to a named object, and the chip returns the secret only if both the endorsement key and that object reside in it. The exchange is inherently interactive, which is why an issuer exists at all.

B. WebAuthn's requirement

A WebAuthn [8] `tpm` attestation statement requires `certInfo` to be signed by the key in `x5c[0]`. An endorsement certificate cannot occupy that slot, both because its key cannot sign and because it is public data anyone who has seen the machine can copy. The slot requires an attestation-key certificate, and that is what a chip in the 26.8% cannot obtain.

III. Problem and threat model

We want a verifier, with no online party to ask, to conclude that a given attestation key resides in a chip a named silicon vendor certified.

The adversary is computationally bounded, controls every host it operates including their operating systems, may create unlimited network identities and key pairs at negligible cost, may observe all published state, and may participate in the protocol in any role, including as a challenger.

We assume it cannot (i) obtain a silicon vendor's attestation signing key or induce it to certify a key of the adversary's choosing, or (ii) cause a genuine secure element to attest to a key that does not reside in it.

We deliberately do *not* assume the adversary cannot extract a private key from a chip it physically holds. Published attacks do exactly that [18], [19], and §V-D shows the construction does not need the assumption. Assumption (i) is the load-bearing one and §IX is about it.

We assume an **ordering service**: a facility that publishes messages in a total order all verifiers agree on, where "strictly later" is well defined and positions are not forgeable. A blockchain provides this; so does an append-only transparency log or a BFT broadcast service. We assume nothing about its confidentiality—every message below is public—and nothing about honest participation beyond the challenger condition stated in §IV-D.

We assume verifier clocks do not agree, and evaluate all certificate validity at an ordering-agreed anchor time.

IV. Construction

A. Replacing the issuer with an ordering

An issuer converts the interactive proof into a durable artifact by signing it. That works, and it is expensive in a sense unrelated to computation: a long-lived key able to assert any endorsement identity is a key able to mint identities, and it must be guarded as such for the lifetime of the system.

Observation 1 (Replayable challenges): The credential blob returned by `TPM2_MakeCredential` is a deterministic function of the triple (seed, object name, secret). Only the OAEP-wrapped seed is randomised, and no verifier requires that half.

This follows from the credential-protection construction of TPM 2.0 Part 1 §24 [10]: the blob is a symmetric encryption of the secret under a key derived from (seed, name) by KDFa, with an HMAC over the ciphertext and the name under a second derivation from the same pair. Every input is fixed by the triple. We confirmed it empirically: two calls with the same seed produce identical blobs and different wrapped seeds.

Observation 1 is what removes the authority. A challenge issued once can be recomputed by every verifier afterwards, offline and indefinitely, from values that are public after the fact. The interactive proof does not need to be signed into an artifact; it needs only to be *ordered*.

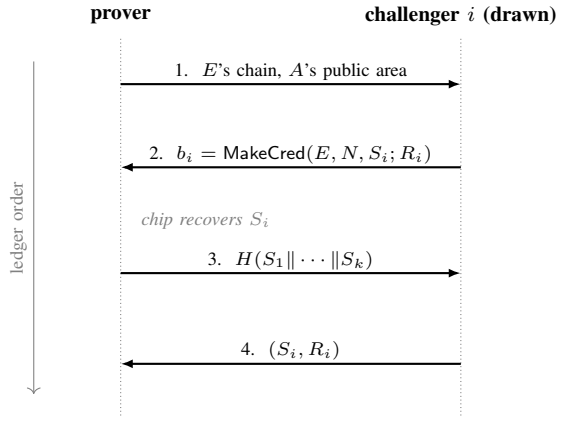

Fig. 1. The four messages. Each is published strictly later in the ledger order than the message it answers; steps 2 and 4 occur once per drawn challenger. Nothing is signed, and the ordering, not any signature, is what makes the exchange a proof (§V).

B. The protocol

Notation. E is a chip's endorsement key and N the object name of an attestation key A in the same chip, N being a digest of A's public area. k is the number of challengers, S_i the secret challenger i seals and R_i the seed it seals under, b_i the resulting credential blob, and H a collision-resistant hash. *Order* is the total order of §III; "strictly later" excludes equal positions.

Definition 1 (CA-free enrolment): An enrolment is the following sequence of published messages, each strictly later in the order than the message it answers:

1) the prover publishes E's certificate chain and A's public area, from which N is derived;
2) each of k drawn challengers publishes $b_i = \texttt{MakeCredential}(E, N, S_i; R_i)$ together with the wrapped seed;
3) the prover publishes $H(S_1 \| \cdots \| S_k)$, having recovered every S_i by `TPM2_ActivateCredential`;
4) each challenger publishes (S_i, R_i).

Verifiers accept when every b_i is reproduced by (S_i, R_i) under (E, N) and the commitment matches the concatenation in canonical challenger order.

Figure 1 shows the exchange. Nothing is signed and no key persists between enrolments: there is nothing to steal, to rotate, or to guard, and no party holds an authority another must trust.

C. What the verifier checks

The endorsement chain must reach a pinned vendor root, every signature verifying at the anchor time; the leaf must carry the `tcg-kp-EKCertificate` extended key usage, must not be a CA, and must be encipherment-only. A certificate claiming signing capability is not an endorsement key, and treating one as such would undermine the reason the exchange exists. Client-supplied intermediates are a routing hint: every link is verified and only pinned roots are trust input.

The attestation key's public area must be `restricted`, `sign`, not `decrypt`, `fixedTPM`, `fixedParent`, `sensitiveDataOrigin`, and must declare a real signing scheme rather than `TPM_ALG_NULL`. §V explains why each is load-bearing.

D. Challenger selection

The residual attack is a challenger privately leaking S_i to a prover holding no chip. The defence is not to trust challengers but to require several and to *draw* rather than accept them: the prover must recover every S_i, so forgery requires all k to collude. This replaces a key someone promises to protect with a condition the system can observe.

The draw is weighted by a quantity the adversary cannot manufacture cheaply and keyed on a beacon unpredictable before the enrolment was published. An unweighted draw over a permissionless set is the obvious error: cheap identities crowd the ballot and an adversary able to mint them draws its own challengers, restoring the attack in full. The draw is without replacement, since seating one challenger twice halves the collusion forgery requires, and a short set is refused rather than accepted—an adversary able to shrink the challenger pool must not thereby reduce what forgery costs.

E. Resampling the draw

The collusion requirement of §IV-D is only half of the property. The other half is *being drawn*, and an adversary can retry that.

An enrolment that does not complete must eventually be abandonable, or a prover whose drawn challengers are unreachable is excluded permanently through no fault of its own—a liveness requirement, and in deployment a pressing one, since a challenger running older software answers nothing and cannot be distinguished from one that is merely slow. Abandonment means republication, and republication at a new height draws a new set. Nothing then prevents an adversary from republishing until the draw seats only its own nodes.

Let the adversary hold a fraction p of the draw weight and let W be the enrolment window. Forgery per attempt has probability approximately p^k for small k, and attempts are unbounded at one per W, so the expected time to a forged enrolment is about W/p^k rather than the ∞ implied by "forgery requires all k to collude". Simulated against a deployed weight distribution of 13 candidates with $k = 3$, weighted and without replacement:

adversary weight	Pr[forge] per window	expected time
25.5%	0.53%	≈64 h
31.4%	1.54%	≈22 h
36.4%	2.91%	≈12 h
44.4%	6.59%	≈5 h

At $W = 180$ blocks (≈20 minutes) an adversary holding roughly a third of the draw weight forges in half a day. The stake is rented rather than burned, so this is a cost of capital for the duration, not a loss. Each success yields one identity per endorsement certificate held; endorsement certificates are public and copyable, so an adversary with a harvested set converts grinding time into identities at that rate.

Two things bound it in practice and neither is a defence. The figures are dominated by the small candidate set: the same adversary share against a few hundred challengers is a different problem, so this is a small-network weakness—which is precisely when it is cheapest to exploit and least likely to be noticed. And the adversary must run bonded infrastructure that participates honestly in consensus meanwhile.

We record this as an unresolved tension rather than a solved problem, because the obvious fixes trade against the liveness requirement that created it. Fixing the set at first draw and re-drawing only the slots that did not answer preserves grinding resistance but needs a way to distinguish a silent challenger from a slow one. Charging an escalating fee per republication makes grinding pay super-linearly while one honest retry stays cheap, at the cost of pricing a remedy that exists for blameless provers. Raising k or growing the candidate set moves the exponent and the base, and is the only direction that helps without a trade—but both are properties of a deployment's size rather than of the construction.

F. Freshness

An enrolment records that a key lives in a certified chip; it says nothing about when, and a proof from last year is not evidence the machine still exists. Each subsequent use therefore presents a fresh `TPM2_Certify` under the enrolled key whose `extraData` carries a challenge bound to that use. The verifier additionally checks the `TPM_GENERATED` magic, the `TPM_ST_ATTEST_CERTIFY` structure type—a different type places different fields at the same offsets, so accepting one would mean parsing some other structure as this one—and that the certified object is the enrolled key rather than another object in the chip.

V. Security analysis

A. The ordering is the proof

Claim 1: The equations of Definition 1, verified without regard to publication order, are satisfiable by a prover holding no TPM.

Such a prover chooses S_i and R_i itself and computes b_i from the endorsement public key, which is public. Every check passes: each b_i is reproduced by its (S_i, R_i), and the commitment matches by construction. Presented with all four messages simultaneously, a verifier cannot distinguish this from a genuine enrolment.

What separates them is that message 3 was published after every message 2 and before every message 4. The prover cannot learn S_i except by holding the chip, and must commit before the reveal, so it cannot read the answer off the record. Message 2 cannot precede message 1, because `MakeCredential` seals to a specific object name, which is a digest of the public area published in message 1. The four messages are minimal: no adjacent pair may be merged.

Two consequences are easy to implement incorrectly. First, equal positions are not ordered: two messages at the same position have no order verifiers agree on, so every comparison must be strict. Second, the commitment must be over all k secrets jointly—one commitment per secret would let a prover answer whichever challenges it managed to open and abandon the rest, which is the k-of-k requirement dissolving into 1-of-k.

TABLE I
Attacks and the mechanism that refuses them

Attack	Refused by
Replay a copied endorsement certificate	Possession proof: the seed unwraps only under the endorsement private key
Prover seals its own challenge	Challenger set is drawn, not chosen (§IV-D)
Prover commits after seeing a reveal	Strict ordering, message 3 before 4
Prover opens some challenges, guesses the rest	Joint commitment over all k secrets
Challenger reveals a different secret	Blob already published; MakeCredential is deterministic
Enrol a second key for a second identity	Binding is on the endorsement key
Reuse an old certify	`extraData` carries the current challenge
Certify some other object in the chip	Certified name must equal the enrolled name
Enrol a key that can sign arbitrary data	`restricted` required (Claim 2)
Duplicate an enrolled key to another chip	`fixedTPM`, `fixedParent` required
Extract a key from a chip in hand	*Not refused; see §V-D*
All k challengers collude	**Not refused**
Vendor CA compromise	**Not refused** (§IX)

B. The attribute the argument rests on

Claim 2: If an unrestricted signing key may be enrolled, the certify carries no information about where a key resides.

A `restricted` signing key signs only structures the TPM itself generated. An unrestricted key signs whatever its host composes, including a forged `TPMS_ATTEST` certifying a key that never existed in any chip; a verifier would then accept the forgery under a genuinely enrolled name. The public area is therefore validated before any challenge is issued, and must further carry `sensitiveDataOrigin`—absent it, the private half may have been generated outside and imported—and `fixedTPM` with `fixedParent`, absent which the key is duplicable to another chip and one enrolment would licence every machine it is copied to.

C. Attacks considered

Collusion of all k drawn challengers admits a prover with no chip. We do not claim otherwise; the design intent is that this is a condition the system can *observe*—the drawn set is public, weighted and beacon-keyed—rather than a key whose custody someone asserts. With $k = 3$, the smallest value at which no single challenger can admit provers alone, an adversary must both control challengers and win the draw for the specific enrolment it wishes to forge.

D. What key extraction buys

It is tempting to treat secure-element key extraction as the assumption such schemes live or die by. It is not, and the reason is worth stating precisely, because the contrary intuition is strong.

Suppose the adversary extracts the endorsement private key from a chip it physically holds. It can now attest as that chip without the chip. But the identity a verifier derives is a digest of that chip's endorsement public key—one fixed value—so the adversary holds exactly the one identity it already had by owning the chip. Enrolling many attestation keys against the extracted endorsement key changes nothing: they all resolve to the same endorsement identity.

Now suppose the technique *generalises across a chip family*. This is the case usually described as catastrophic, and it still does not multiply identities. To present identity j the adversary needs chip j's private key *and* chip j's vendor-signed certificate over the matching public key. Extraction supplies the first for chips in hand; nothing supplies the second. Certificates over keys the vendor never certified require the vendor's signing key, which is assumption (i)—a different assumption, and the subject of §IX.

What extraction genuinely buys is a lower price floor. An extracted credential is a file: it does not need the chip powered, present, or working. A deployment that prices identity in hardware should therefore price it in *obtainable chips*, including scrapped ones, rather than in working machines. That is a real reduction and a constant factor, not a break.

VI. Implementation

The reference implementation is dependency-free: KDFa, AES-CFB, MGF1, RSA-OAEP, PKCS#1 v1.5 verification and the DER encoding are written out rather than imported. A verifier that must agree byte-for-byte with every other verifier is poorly served by whichever version of a library each host happens to have installed. Two obstacles cost us time and will cost others the same.

A. Real endorsement certificates are not strictly DER

AMD encodes `critical: FALSE` explicitly where DER requires the default omitted. Python's `cryptography` refuses such a certificate outright with an `EncodedDefault` parse error, while `openssl` and Rust's `x509-parser` accept it. An implementer reaching for the obvious library will conclude the hardware is defective; it is not, and the certificate is the one the vendor issued. Endorsement-chain parsing must use a lenient parser, shared by all verifiers.

B. A published "root" that is an intermediate

The certificate one vendor publishes as its endorsement root is cross-signed by a commercial certificate authority and is therefore an intermediate. Pinning it would silently relocate the trust decision to that authority, which is a different assertion from "this vendor manufactured this chip". Verify self-signedness before pinning; we omit that vendor rather than pin an intermediate.

C. Signature verification

PKCS#1 v1.5 verification must compare the entire re-encoded block [15]. An implementation that instead searches for the digest within the padding accepts trivial forgeries against small exponents—Bleichenbacher's 2006 result [16], [17], which production libraries have shipped more than once.

VII. Evaluation

We exercised the construction against a TPM 2.0 simulator, driving our own command encodings. The TCG endorsement template derives its 314-byte public area; a restricted RSA-2048 signing key derives; our challenger's credential blob is accepted and opened by `ActivateCredential`, returning the sealed secret; an independent verifier re-derives the identical blob from the revealed pair; and a certify verifies, while a certify produced by a *different* key in the same chip is refused under the enrolled key's public area. The full four-message enrolment reaches its proven state on values the chip actually produced.

A. On physical silicon

Simulation cannot establish that vendor silicon behaves this way: a simulator accepts templates a firmware TPM may reject and carries no endorsement certificate at all, so the vendor signature—the thing the construction ultimately rests on—cannot be exercised there. We therefore completed the enrolment end to end on an AMD firmware TPM (version 3.92.0.5), on a live permissionless chain, with challengers drawn from its participants rather than from a harness.

Four observations are worth reporting because each contradicted an expectation.

The endorsement certificate was not where the profile puts it. Both standard NV indices were empty, as were all four platform-crypto-provider properties. The certificate existed only in an operating-system store, as a 1,322-byte structure of property records in which the certificate is one tagged field rather than the whole value. A reader following the TCG profile concludes the machine has no certificate and refuses a chip that is sound.

The chain existed only over AIA. Neither intermediate was present in any system certificate store; each had to be fetched from the authority-information-access extension of the certificate below it, and the leaf is published at no vendor URL—it exists on the machine or not at all. This asymmetry matters for platform coverage: where an operating-system store can recover a certificate the chip does not expose, a platform without one has no recourse.

The endorsement key refused to act as a storage parent. `TPM2_Create` beneath it returns `TPM_RC_HANDLE`, rejected at handle validation. This forecloses on hardware a design that is independently unsound in theory: certifying an attestation key by *descent* from the endorsement key, and shipping one self-contained blob, fails because `parentName` is a field inside a structure signed by the claimant—a software key reproduces it—and on this silicon the attempt does not even reach that objection.

Opening the credential cost nothing. `TPM2_ActivateCredential` against an `adminWithPolicy` endorsement key, authorised

through a `PolicySecret` session on the endorsement hierarchy, succeeded and consumed *zero* dictionary-attack attempts: the lockout counter stood at 0 of 32 before and after four rounds opening three credentials each. This could not be known in advance and it bounds the retry economics: a counter that advanced per attempt would cap retries and make every failed enrolment expensive.

B. A root set is never finished

Deployment surfaced a failure mode the construction does not have but every instance of it does. A vendor ships more than one endorsement family: pinning the root that one product line chains to silently refuses every machine on the other. In our deployment a current laptop from a pinned vendor was rejected as uncertified because that vendor operates two endorsement roots and only one had been pinned; the chain was sound and the omission was ours.

This is not a detail of operations. The verifier's answer to "did a vendor certify this chip" is exactly as complete as its root set, an incomplete set is indistinguishable from a forged chain at validation time, and the failure is silent and biased—it excludes honest hardware while costing an adversary nothing. Since determinism forbids fetching roots at validation time (§IX), the set is a consensus parameter, and enlarging it is a gated protocol change rather than a configuration edit. An implementation should therefore *retain the chains it refuses*, so that a missing vendor is a filed artefact rather than an anecdote from whoever happened to be excluded.

VIII. Application: admission control

The construction was built for a concrete need, and the need illustrates what it is good for.

A permissionless system that rewards *participants* rather than capital must decide who counts as one. Every per-identity admission rule—registration work, waiting periods, address caps, probation—shares a defect.

Observation 2 (Linearity): Let c be the cost such a rule imposes per identity and r the reward per identity. An adversary controlling n identities pays nc and earns nr; an honest participant pays c and earns r. The cost-to-reward ratio is c/r for both, so raising c does not disadvantage the adversary—it prices out the honest participant first, since the adversary amortises fixed setup across n.

On the deployment we report, four such rules were tried in succession and all four failed: on 2026-09-06, approximately $1{,}000$ of $1{,}191$ registered identities belonged to two or three operators running headless browser farms, capturing roughly 42% of emission.

Hardware attestation is a natural response, and it is important to be exact about what it does, because the obvious claim—that it makes identities *scarce* rather than merely expensive—is false.

Observation 3 (Device gating is also linear): A device costs money. An adversary seeking n identities buys n devices and pays n times the unit price, for n times the reward. Observation 2 applies unchanged: the cost-to-reward ratio is identical for the adversary and the honest participant, and device gating is a capital gate denominated in hardware rather than in tokens.

TABLE II
Attestation attempts by outcome ($n = 664$)

Outcome	Share
Succeeds (phone / hardware wallet)	47.9%
TPM healthy, platform will not attest	$\mathbf{26.8}\%$
User-fixable (password manager took the ceremony)	18.8%
Succeeds (Windows Hello attests)	6.5%

A farm of cheap attesting devices is the ASIC of this mechanism, and nothing in the construction prevents one. We do not claim a categorical difference from capital, because there is none.

What changes is the *constant*, and in the measured case the constant was the entire problem. The attack we observed was not capital-intensive: it was browser processes on rented servers, where the marginal cost of the thousandth identity was indistinguishable from zero. Device gating moves that marginal cost from approximately nothing to the price of an obtainable chip. That is a magnitude, not a principle, and magnitudes decide whether an attack is worth running.

Two further properties are real but should not be oversold. The identity binds to the *endorsement* key, which is one per chip by manufacture, so a chip that enrols ten attestation keys still holds one identity and regenerating the endorsement seed invalidates the vendor certificate. And the cost is a capital good rather than a burn: unlike Proof of Work's energy, a device retains resale value, which makes a device farm cheaper in net terms than its sticker price suggests.

The one genuinely non-linear ingredient available here is human attention. A WebAuthn ceremony requires a user gesture per signature, and human attention does not scale with capital the way hardware does—which is the sense in which the mobile path prices an identity in something an adversary cannot simply buy more of. §IV deliberately gives that up: automatic enrolment is what lets a headless machine participate at all, and it returns the mechanism to a pure capital gate with a high unit floor. A deployment that wants the non-linear ingredient must keep a ceremony; this construction is for the deployments that cannot.

In this setting the ordering service is the chain itself, challengers are drawn from bonded validators weighted by stake and keyed on the beacon that keys producer selection, and the four messages are ordinary transactions. Admission is consensus input, so verification must be a pure function of agreed data: roots are pinned constants never fetched at validation time, revocation is deliberately not consulted—it would make validation network-dependent, and a compromised intermediate is handled as a root-set change—and certificate validity is evaluated at an anchor block's timestamp rather than a wall clock, since a certificate expiring mid-block would otherwise be valid on one verifier and expired on the next.

One property is worth noting because it is usually a cost of attestation-based admission and is absent here. Schemes built on mobile key attestation exclude devices whose owners have unlocked them [9], which falls hardest on the users most committed to controlling their own machines. Boot state is a mobile key-attestation property; a TPM's endorsement key is fixed at manufacture and independent of what the machine boots, so an owner running a self-built kernel with no secure boot enrols on the same terms as anyone else. For the Sybil question this is not a concession: a modified host running a genuine chip is still one chip and therefore one identity.

Which roots are pinned is therefore the operative dial—it sets the unit price of Observation 3—and it reverses only in the widening direction, since narrowing later strands everyone already enrolled. A deployment pricing identity in hardware should price it in *obtainable* chips, including scrapped ones (§V-D), not in working machines.

IX. Open problem: vendor CA integrity

Everything above reduces to one assumption, and we cannot discharge it.

> A silicon vendor's attestation signing key is not compromised, and the vendor does not certify keys that are not resident in genuine secure elements it manufactured.

This is the only assumption whose failure is a *break* rather than a price. A compromised or misused vendor signing key mints unlimited (key, certificate) pairs, each with a distinct endorsement public key and therefore a distinct identity. Uniqueness enforcement sees nothing wrong: every one of them is a different chip as far as any verifier can tell. Where the analysis of §V-D shows that holding chips bounds an adversary, holding the vendor's key removes the bound entirely.

Nothing in the protocol detects this. A verifier checks that a chain reaches a pinned root; a forged-but-validly-signed chain reaches it. Where revocation is not consulted—which determinism requires in a replicated setting—even a vendor that discovered the compromise could not stop acceptance at validation time; the response is a root-set change, with governance latency.

We see three directions and have implemented none.

Per-issuer issuance observability. A leaked vendor key used at scale produces an anomalous burst of first-time identities under one root or one issuing intermediate. That is measurable from published state alone, deterministically, and could bound admission rate per *issuer* rather than per identity. Such a rule is not subject to Observation 2, because the adversary cannot mint issuers. We regard this as the most promising direction.

Multi-vendor corroboration. Requiring chains under two independent roots would make a single vendor compromise insufficient. It is unimplementable on current hardware: a chip carries one endorsement certificate from one manufacturer.

Transparency logs. Certificate Transparency's structure applies—an append-only log of issued endorsement certificates would make mass issuance visible—but no vendor operates one.

We state this as the open problem rather than distributing it through a costs section, because it is the one place where hardware-anchored identity is strictly worse than Proof of Work: an adversary cannot forge hashrate, and this adversary would need exactly one key.

X. Related work

Attestation without an issuer. Direct Anonymous Attestation [12] addresses the adjacent problem—removing the privacy CA's ability to link a chip's attestations—using group signatures with an issuer that establishes the group. It provides unlinkability, which we do not, and requires an issuer and its setup, which we do not. Our construction removes the issuing party rather than blinding it, at the cost of interaction and of a public ordering; it is applicable precisely because an ordering service supplies that ordering, and would not be applicable without one. The two are complementary: a deployment needing unlinkability should use DAA, and one needing no issuer should use this.

Remote attestation generally. SGX remote attestation [13] likewise terminates in a vendor service and shares the failure mode we measured: when the service is unavailable to a party, the hardware's evidence becomes unusable regardless of its integrity.

Secure-element key extraction. TPM-FAIL [18] recovered ECDSA keys from an Intel firmware TPM and an STMicroelectronics TPM by timing analysis; faulTPM [19] extracted secrets from AMD firmware TPMs by voltage fault injection; Shakevsky et al. [20] showed a mobile keystore permitting attestation-relevant compromise by design error. §V-D argues these bound a price rather than breaking the construction, and we would welcome disagreement.

Sybil resistance. Douceur [2] establishes that without a trusted identity authority an adversary can present arbitrarily many identities absent a resource constraint; §VIII is a choice of which resource. Social-graph defences [3] bound identities by trust-graph structure. Proof-of-personhood [4], [5] and synchronous ceremonies [7] bind identities to physical presence, proving more than hardware attestation does at the cost of a ceremony every participant must attend.

Commit–reveal. The commitment scheme is textbook [14]; the contribution is Observation 1, that a TPM credential blob is deterministic in its inputs, which is what makes a commit–reveal sufficient here. Without it a challenge could not be replayed and every verifier would need the challenger's word.

XI. Conclusion

A TPM's endorsement key is vendor-certified and cannot sign; the key that signs an attestation must therefore be certified by someone, and that someone has always been an issuer. We show the issuer is not necessary. Because a TPM credential blob is deterministic in its inputs, an interactive possession proof can be replayed offline by every verifier forever, and what an issuer's signature was providing—a durable, checkable record of an exchange that happened—is supplied instead by the order in which four public messages were published.

The security of the result rests on publication order and on one TPM object attribute, both of which a verifier checks itself, and on a challenger set drawn rather than chosen. It does not rest on secure-element key extraction being infeasible: extraction yields one identity per chip physically obtained however far the technique generalises, because certifying a key the vendor never certified requires the vendor. It does rest, entirely, on vendor certificate authority integrity—and that, rather than anything about chips or ceremonies, is where we think the attention belongs.

We are correspondingly careful about what the admission application claims. Device gating is a capital gate with a high unit floor, not an escape from the linearity that defeats per-identity rules; a device farm is its ASIC, and pricing one is an economic question rather than a security property. What it removed, in the deployment we measured, was a regime where the marginal identity cost nothing at all.

AVAILABILITY

The reference implementation, specification, pinned roots and tests are public at `https://github.com/hclivess/posea`: the enrolment of §IV is `posea/tpm.py` and `posea/enrolment.py`, specified normatively in `SPEC.md` §9. The deployment of §VIII is at https://github.com/hclivess/nado. The reverse-engineered platform key-attestation claim format, which is what one must decode to establish that the platform path cannot be repaired from the client side, is documented at https://github.com/hclivess/windows-tpm-claim-format.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[2] J. R. Douceur, "The Sybil attack," in *Proc. 1st Int. Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.

[3] H. Yu, M. Kaminsky, P. B. Gibbons, and A. Flaxman, "SybilGuard: Defending against Sybil attacks via social networks," in *Proc. ACM SIGCOMM*, 2006.

[4] M. Borge, E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, and B. Ford, "Proof-of-personhood: Redemocratizing permissionless cryptocurrencies," in *IEEE European Symp. on Security and Privacy Workshops (EuroS&PW)*, 2017.

[5] B. Ford and J. Strauss, "An offline foundation for online accountable pseudonyms," in *Proc. 1st Workshop on Social Network Systems (SocialNets)*, 2008.

[6] V. Buterin and V. Griffith, "Casper the friendly finality gadget," arXiv:1710.09437, 2017.

[7] Idena, "Idena: Proof-of-person blockchain." [Online]. Available: https://idena.io

[8] W3C, "Web Authentication: An API for accessing public key credentials," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/webauthn-3/

[9] Google, "Verifying hardware-backed key pairs with key attestation," Android Developers. [Online]. Available: https://developer.android.com/privacy-and-security/security-key-attestation

[10] Trusted Computing Group, "Trusted Platform Module Library, Part 1: Architecture" and "Part 3: Commands," Family 2.0, Level 00, Rev. 01.59, 2019.

[11] Trusted Computing Group, "TCG EK Credential Profile for TPM Family 2.0," Level 0, Version 2.5.

[12] E. Brickell, J. Camenisch, and L. Chen, "Direct anonymous attestation," in *Proc. 11th ACM Conf. on Computer and Communications Security (CCS)*, 2004, pp. 132–145.

[13] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptology ePrint Archive, Report 2016/086, 2016.

[14] M. Blum, "Coin flipping by telephone: A protocol for solving impossible problems," in *Proc. CRYPTO*, 1981, pp. 11–15.

[15] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC 8017, IETF, 2016.

[16] H. Finney, "Bleichenbacher's RSA signature forgery based on implementation error," openssl-dev mailing list, Aug. 2006.

[17] "CVE-2006-4339: OpenSSL RSA signature forgery," 2006.

[18] D. Moghimi, B. Sunar, T. Eisenbarth, and N. Heninger, "TPM-FAIL: TPM meets timing and lattice attacks," in *Proc. 29th USENIX Security Symp.*, 2020.

[19] H. N. Jacob, C. Werling, R. Buhren, and J.-P. Seifert, "faulTPM: Exposing AMD firmware TPMs," arXiv:2304.14717, 2023.

[20] A. Shakevsky, E. Ronen, and A. Wool, "Trust dies in darkness: Shedding light on Samsung's TrustZone keymaster design," in *Proc. 31st USENIX Security Symp.*, 2022.

[21] J. Kučera, "Windows TPM key-attestation claim format," 2026. [Online]. Available: https://github.com/hclivess/windows-tpm-claim-format